204252 Data Structures and Analysis

Chapter 11: Graphs (Part 1)

Lecturer

August 31, 2026

Computer Science Department

Outline

Graph Terminology & Definitions

Graph Representations

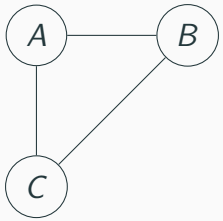
Graph Traversals: BFS & DFS

Graph Terminology & Definitions

Definitions of Vertices and Edges

A **Graph** $G = (V, E)$ consists of:

- V: A finite set of **vertices** (or nodes).
- E: A set of **edges** (or arcs) connecting pairs of vertices. Each edge is a pair (u, v) where $u, v \in V$.



Graph Classification

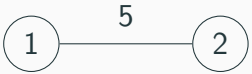
- **Directed Graph (Digraph)**: Edges are ordered pairs (u, v) with direction from $u \to v$. Vertex v is adjacent to u, but u is not necessarily adjacent to v.
- **Undirected Graph**: Edges (u, v) have no direction. If $(u, v) \in E$, then $(v, u) \in E$ implicitly.
- **Weighted Graph**: Each edge has an associated numerical value (weight or cost).



Undirected



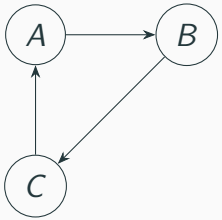
Directed



Weighted

Degrees in Graphs

- **Degree** of a vertex v (undirected graph): The number of edges incident on v.
- **In-Degree** of v (directed graph): The number of incoming edges pointing to v.
- **Out-Degree** of v (directed graph): The number of outgoing edges starting from v.



For Node A:
- In-degree $= 1$ (from C)
- Out-degree $= 1$ (to B)
- Total degree $= 2$

Paths and Cycles

- **Path**: A sequence of vertices $w_1, w_2, \ldots, w_N$ such that $(w_i, w_{i+1}) \in E$.
- **Simple Path**: A path in which all vertices are distinct (except possibly the first and last).
- **Path Length**: The number of edges in the path (equal to $N - 1$).
- **Cycle**: A path of length ≥ 1 such that $w_1 = w_N$.
- **Simple Cycle**: A cycle that is simple. For undirected graphs, the edges must be distinct.
- **Directed Acyclic Graph (DAG)**: A directed graph containing no cycles.

Connectivity & Complete Graphs

- **Connected (Undirected)**: An undirected graph is connected if there is a path from every vertex to every other vertex.
- **Strongly Connected (Directed)**: A directed graph is strongly connected if there is a path from every vertex to every other vertex.
- **Weakly Connected (Directed)**: A directed graph is weakly connected if its underlying undirected graph (ignoring directions) is connected.
- **Complete Graph (K_n)**: A graph in which there is an edge between every pair of vertices. An undirected complete graph has $|V|(|V|-1)/2$ edges.

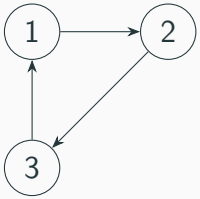
Graph Representations

Adjacency Matrix Representation

An **Adjacency Matrix** is a 2D array A of size $|V| \times |V|$:

$$A[i][j] = \begin{cases} 1 & \text{if there is an edge } (i, j) \\ 0 & \text{otherwise} \end{cases}$$

If weighted, set $A[i][j] = \text{weight}$ (and ∞ or 0 for no edge).



$$\begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

Space complexity: $\mathcal{O}(|V|^2)$

Pros: $\mathcal{O}(1)$ edge lookups.

Cons: Wastes space for sparse graphs.

Adjacency Matrix C++ Implementation

```cpp
class AdjacencyMatrixGraph {
private:
    int numVertices;
    vector<vector<int>> adjMatrix;

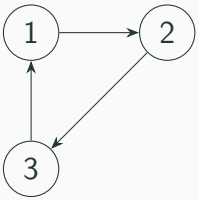
public:
    AdjacencyMatrixGraph(int vertices) : numVertices(vertices) {
        adjMatrix.resize(vertices, vector<int>(vertices, 0));
    }

    void addEdge(int u, int v, int w = 1) {
        adjMatrix[u][v] = w; // For directed graphs
    }

    bool hasEdge(int u, int v) {
        return adjMatrix[u][v] != 0;
    }
};
```

Adjacency List Representation

An **Adjacency List** uses an array of linked lists (or vectors) of size $|V|$. For each vertex u, the list contains its adjacent vertices.



Lists:

`Adj[1]` $\rightarrow$ `[2]`

`Adj[2]` $\rightarrow$ `[3]`

`Adj[3]` $\rightarrow$ `[1]`

Space complexity: $\mathcal{O}(|V| + |E|)$

Pros: High space efficiency for sparse graphs.

Cons: $\mathcal{O}(\text{degree}(u))$ to look up an edge.

Adjacency List C++ Implementation

```cpp
class AdjacencyListGraph {
private:
    int numVertices;
    vector<vector<int>> adjList;

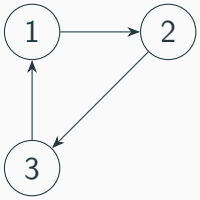
public:
    AdjacencyListGraph(int vertices) : numVertices(vertices) {
        adjList.resize(vertices);
    }

    void addEdge(int u, int v) {
        adjList[u].push_back(v); // For directed graphs
    }

    bool hasEdge(int u, int v) {
        for (int neighbor : adjList[u]) {
            if (neighbor == v) return true;
        }
        return false;
    }
};
```

Edge List Representation

An **Edge List** represents a graph as a collection (e.g., list, array, or vector) of all its edges. Each edge is typically stored as a pair/tuple of vertices or a custom structure.



Edge List:

`Edges = [ (1, 2), (2, 3), (3, 1) ]`

Space complexity: $\mathcal{O}(|E|)$

Pros: Simple and extremely memory-efficient if we only store edges. Ideal for edge-centric algorithms (e.g., Kruskal's MST, Bellman-Ford).

Cons: Finding neighbors or checking if an edge exists requires a full scan of all edges.

Edge List C++ Implementation

```cpp
struct Edge {
    int src;
    int dest;
    int weight; // Optional, if graph is weighted
};

class EdgeListGraph {
private:
    int numVertices;
    vector<Edge> edges;

public:
    EdgeListGraph(int vertices) : numVertices(vertices) {}

    void addEdge(int u, int v, int w = 1) {
        edges.push_back({u, v, w});
    }

    bool hasEdge(int u, int v) {
        for (const auto& edge : edges) {
            if (edge.src == u && edge.dest == v) return true;
        }
        return false;
    }
};
```

Complexity Comparison

Operation	Adjacency Matrix	Adjacency List	Edge List
Space Complexity	$\mathcal{O}(\|V\|^2)$	$\mathcal{O}(\|V\| + \|E\|)$	$\mathcal{O}(\|E\|)$
Add/Remove Edge	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$ / $\mathcal{O}(\|E\|)$
Check Edge (u, v) Exists	$\mathcal{O}(1)$	$\mathcal{O}(\deg(u))$	$\mathcal{O}(\|E\|)$
Find All Neighbors of u	$\mathcal{O}(\|V\|)$	$\mathcal{O}(\deg(u))$	$\mathcal{O}(\|E\|)$
Iterating Over Whole Graph	$\mathcal{O}(\|V\|^2)$	$\mathcal{O}(\|V\| + \|E\|)$	$\mathcal{O}(\|E\|)$

Note
Most real-world graphs are **sparse** ($|E| \ll |V|^2$), so the adjacency list is the default representation in practice.

Graph Traversals: BFS & DFS

Introduction to Graph Traversals

Goal: Visit all vertices of a graph systematically starting from a source node.
Two Main Approaches:

1. **Breadth-First Search (BFS)**: Explores the graph "level-by-level", visiting neighbors first before going deeper. Uses a **Queue** (FIFO).
2. **Depth-First Search (DFS)**: Explores "deeply" along a path as far as possible before backtracking. Uses a **Stack** (LIFO) or **Recursion**.

Breadth-First Search (BFS) Algorithm

Core Logic:

1. Initialize distance to all vertices as ∞, mark as unvisited.
2. Set source s distance to 0, mark as visited, and push to queue.
3. While the queue is not empty:
   - 3.1 Pop the front vertex u.
   - 3.2 For each unvisited neighbor v of u: set its distance to $\text{dist}[u] + 1$, set its path to u, mark it visited, and push it to queue.

Time Complexity: $\mathcal{O}(|V| + |E|)$ with Adjacency List.
Space Complexity: $\mathcal{O}(|V|)$ to store the queue and visited array.

BFS C++ Implementation

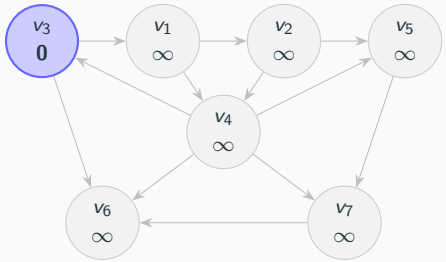
```cpp
void Graph::BFS(int s) {
    vector<int> dist(numVertices, INT_MAX);
    vector<int> path(numVertices, -1);
    vector<bool> visited(numVertices, false);
    queue<int> q;

    dist[s] = 0;
    visited[s] = true;
    q.push(s);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        cout << "Visited node: " << u << " at distance " << dist[u] << "\n";

        for (int v : adjLists[u]) {
            if (!visited[v]) {
                visited[v] = true;
                dist[v] = dist[u] + 1;
                path[v] = u;
                q.push(v);
            }
        }
    }
}
```

BFS Step-by-Step Trace (Initial State)

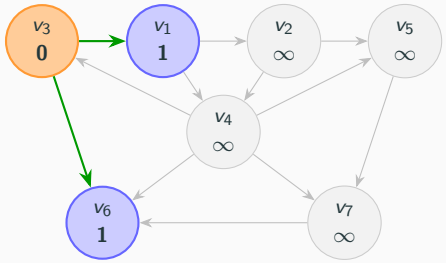


Current Action:
Initialize queue with source v_3.
Distance to v_3 is 0.

Queue Status (FIFO):
`[v3]`

Visited Nodes:
`['v3']`

BFS Step-by-Step Trace (Step 1: Dequeue v3)



Current Action:

Dequeue **v3** and examine neighbors.

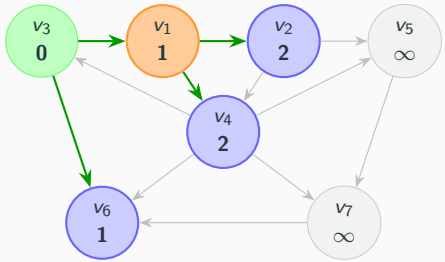
Queue Status (FIFO):

`[v1, v6]`

Visited Nodes:

`['v3', 'v1', 'v6']`

BFS Step-by-Step Trace (Step 2: Dequeue v1)



Current Action:

Dequeue **v1** and examine neighbors.

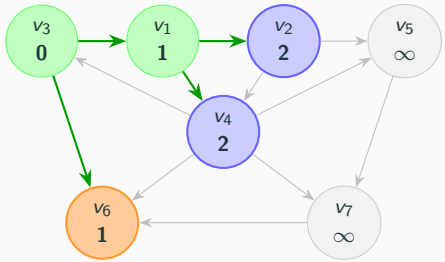
Queue Status (FIFO):

`[v6, v2, v4]`

Visited Nodes:

`['v3', 'v1', 'v6', 'v2', 'v4']`

BFS Step-by-Step Trace (Step 3: Dequeue v6)



Current Action:

Dequeue **v6** and examine neighbors.

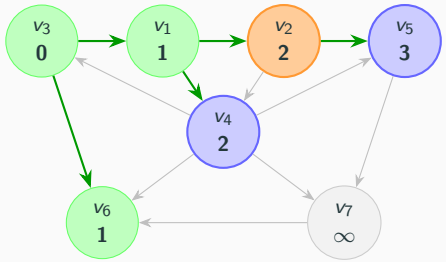
Queue Status (FIFO):

`[v2, v4]`

Visited Nodes:

`['v3', 'v1', 'v6', 'v2', 'v4']`

BFS Step-by-Step Trace (Step 4: Dequeue v2)



Current Action:

Dequeue **v2** and examine neighbors.

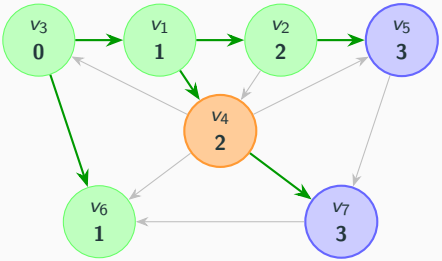
Queue Status (FIFO):

`[v4, v5]`

Visited Nodes:

`['v3', 'v1', 'v6', 'v2', 'v4', 'v5']`

BFS Step-by-Step Trace (Step 5: Dequeue v4)



Current Action:

Dequeue **v4** and examine neighbors.

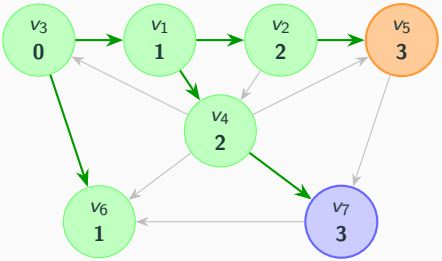
Queue Status (FIFO):

`[v5, v7]`

Visited Nodes:

`['v3', 'v1', 'v6', 'v2', 'v4', 'v5', 'v7']`

BFS Step-by-Step Trace (Step 6: Dequeue v5)



Current Action:

Dequeue **v5** and examine neighbors.

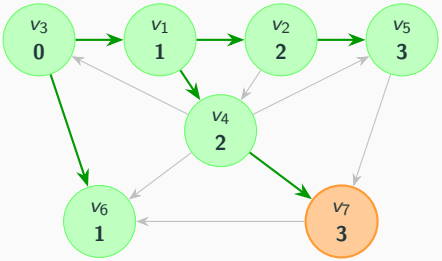
Queue Status (FIFO):

`[v7]`

Visited Nodes:

`['v3', 'v1', 'v6', 'v2', 'v4', 'v5', 'v7']`

BFS Step-by-Step Trace (Step 7: Dequeue v7)



Current Action:

Dequeue **v7** and examine neighbors.

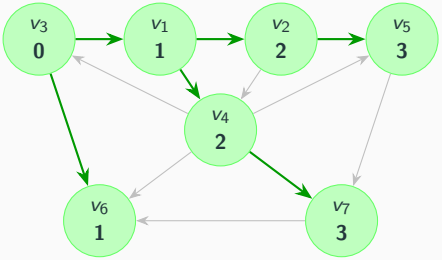
Queue Status (FIFO):

`[]`

Visited Nodes:

`['v3', 'v1', 'v6', 'v2', 'v4', 'v5', 'v7']`

BFS Step-by-Step Trace (Step 8: Queue Empty (Done))



Current Action:
Queue is empty. BFS completes. All reachable nodes have their shortest unweighted path computed.

Queue Status (FIFO):
`[]`

Visited Nodes:
`['v3', 'v1', 'v6', 'v2', 'v4', 'v5', 'v7']`

Depth-First Search (DFS) Algorithm

Core Logic:

1. Initialize all vertices as unvisited.
2. For each vertex u in V:
   2.1 If u is unvisited, call the recursive function `DFS-Visit(u)`.
3. `DFS-Visit(u)`:
   3.1 Mark u as visited.
   3.2 For each neighbor v of u:
   - If v is unvisited, mark edge (u, v) as a **Tree Edge** and recursively call `DFS-Visit(v)`.
   - If v is already visited (active on call stack), mark edge (u, v) as a **Back Edge**.

Complexity: Time $\mathcal{O}(|V| + |E|)$, Space $\mathcal{O}(|V|)$ due to recursion stack.

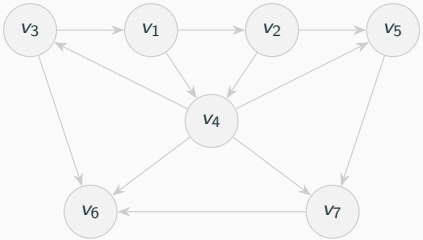
DFS C++ Implementation

```cpp
void Graph::DFSVisit(int u, vector<bool>& visited) {
    visited[u] = true;
    cout << "Visited node: " << u << "\n";

    for (int v : adjLists[u]) {
        if (!visited[v]) {
            DFSVisit(v, visited);
        }
    }
}

void Graph::DFS() {
    vector<bool> visited(numVertices, false);
    for (int i = 0; i < numVertices; i++) {
        if (!visited[i]) {
            DFSVisit(i, visited);
        }
    }
}
```

DFS Step-by-Step Trace (Initial State)

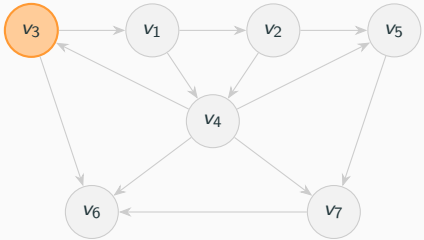


Recursion Call Stack:
Stack is empty. Call `DFSVisit(v3)` to begin.

Visited Nodes:
`[]`

Edge Classifications:
- Green: **Tree Edge** (discovered new node)
- Red/Dashed: **Back Edge** (points to ancestor)

DFS Step-by-Step Trace (Visit v3)



Recursion Call Stack:

v3

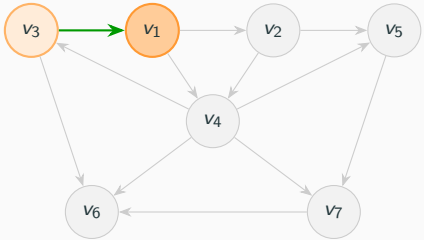
Visited Nodes:

`['v3']`

Edge Classifications:

- Green: **Tree Edge** (discovered new node)
- Red/Dashed: **Back Edge** (points to ancestor)

DFS Step-by-Step Trace (Visit v1)



Recursion Call Stack:

v3 $\rightarrow$ **v1**

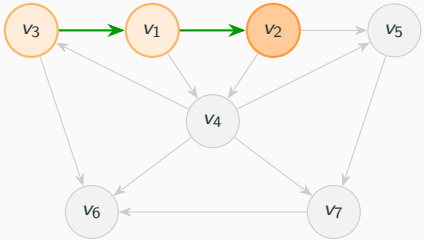
Visited Nodes:

`['v3', 'v1']`

Edge Classifications:

- Green: **Tree Edge** (discovered new node)
- Red/Dashed: **Back Edge** (points to ancestor)

DFS Step-by-Step Trace (Visit v2)



Recursion Call Stack:

`v3` $\rightarrow$ `v1` $\rightarrow$ **v2**

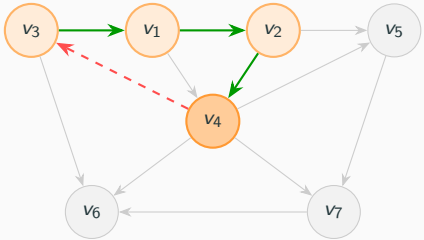
Visited Nodes:

`['v3', 'v1', 'v2']`

Edge Classifications:

- Green: **Tree Edge** (discovered new node)
- Red/Dashed: **Back Edge** (points to ancestor)

DFS Step-by-Step Trace (Visit v4)



Recursion Call Stack:

`v3` $\rightarrow$ `v1` $\rightarrow$ `v2` $\rightarrow$ **v4**

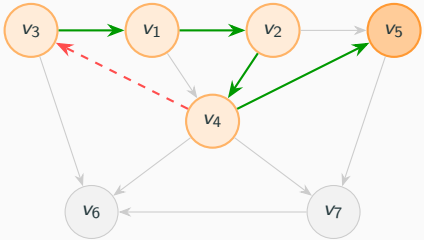
Visited Nodes:

`['v3', 'v1', 'v2', 'v4']`

Edge Classifications:

- Green: **Tree Edge** (discovered new node)
- Red/Dashed: **Back Edge** (points to ancestor)

DFS Step-by-Step Trace (Visit v5)



Recursion Call Stack:

`v3` → `v1` → `v2` → `v4` → **v5**

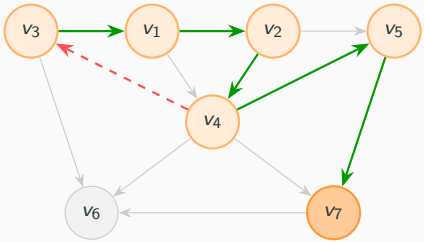
Visited Nodes:

`['v3', 'v1', 'v2', 'v4', 'v5']`

Edge Classifications:

- Green: **Tree Edge** (discovered new node)
- Red/Dashed: **Back Edge** (points to ancestor)

DFS Step-by-Step Trace (Visit v7)



Recursion Call Stack:

`v3` → `v1` → `v2` → `v4` → `v5` → **v7**

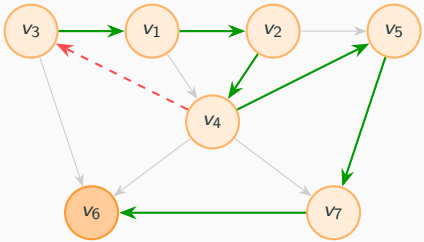
Visited Nodes:

`['v3', 'v1', 'v2', 'v4', 'v5', 'v7']`

Edge Classifications:

- Green: **Tree Edge** (discovered new node)
- Red/Dashed: **Back Edge** (points to ancestor)

DFS Step-by-Step Trace (Visit v6)



Recursion Call Stack:

`v3` → `v1` → `v2` → `v4` → `v5` → `v7` → **v6**

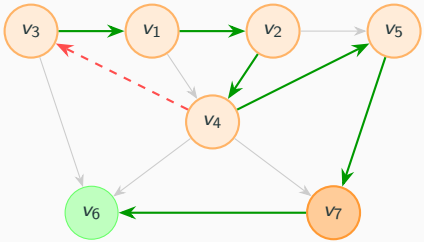
Visited Nodes:

`['v3', 'v1', 'v2', 'v4', 'v5', 'v7', 'v6']`

Edge Classifications:

- Green: **Tree Edge** (discovered new node)
- Red/Dashed: **Back Edge** (points to ancestor)

DFS Step-by-Step Trace (Backtrack from v6)



Recursion Call Stack:

`v3` → `v1` → `v2` → `v4` → `v5` → **v7**

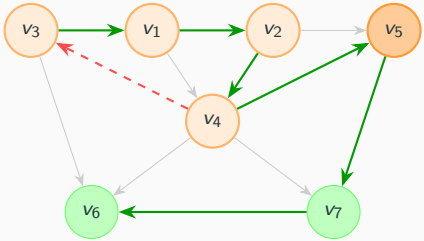
Visited Nodes:

`['v3', 'v1', 'v2', 'v4', 'v5', 'v7', 'v6']`

Edge Classifications:

- Green: **Tree Edge** (discovered new node)
- Red/Dashed: **Back Edge** (points to ancestor)

DFS Step-by-Step Trace (Backtrack from v7)



Recursion Call Stack:

`v3` → `v1` → `v2` → `v4` → **v5**

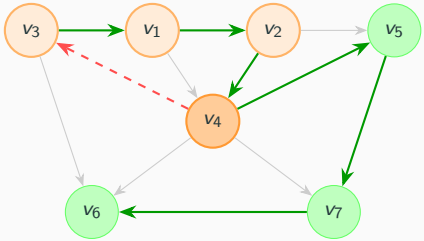
Visited Nodes:

`['v3', 'v1', 'v2', 'v4', 'v5', 'v7', 'v6']`

Edge Classifications:

- Green: **Tree Edge** (discovered new node)
- Red/Dashed: **Back Edge** (points to ancestor)

DFS Step-by-Step Trace (Backtrack from v5)



Recursion Call Stack:

`v3` $\rightarrow$ `v1` $\rightarrow$ `v2` $\rightarrow$ **v4**

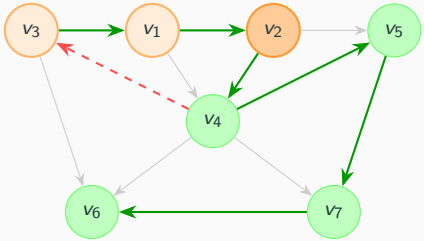
Visited Nodes:

`['v3', 'v1', 'v2', 'v4', 'v5', 'v7', 'v6']`

Edge Classifications:

- Green: **Tree Edge** (discovered new node)
- Red/Dashed: **Back Edge** (points to ancestor)

DFS Step-by-Step Trace (Backtrack from v4)



Recursion Call Stack:

`v3` $\rightarrow$ `v1` $\rightarrow$ **v2**

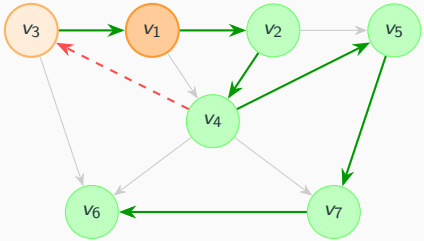
Visited Nodes:

`['v3', 'v1', 'v2', 'v4', 'v5', 'v7', 'v6']`

Edge Classifications:

- Green: **Tree Edge** (discovered new node)
- Red/Dashed: **Back Edge** (points to ancestor)

DFS Step-by-Step Trace (Backtrack from v2)



Recursion Call Stack:

v3 $\rightarrow$ **v1**

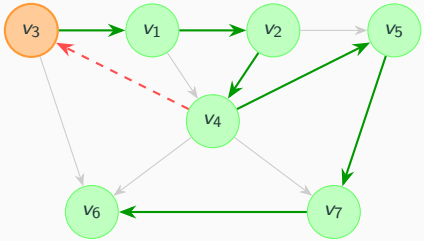
Visited Nodes:

`['v3', 'v1', 'v2', 'v4', 'v5', 'v7', 'v6']`

Edge Classifications:

- Green: **Tree Edge** (discovered new node)
- Red/Dashed: **Back Edge** (points to ancestor)

DFS Step-by-Step Trace (Backtrack from v1)



Recursion Call Stack:

v3

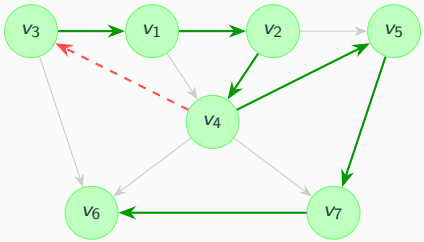
Visited Nodes:

`['v3', 'v1', 'v2', 'v4', 'v5', 'v7', 'v6']`

Edge Classifications:

- Green: **Tree Edge** (discovered new node)
- Red/Dashed: **Back Edge** (points to ancestor)

DFS Step-by-Step Trace (Backtrack from v3 (Done))



Recursion Call Stack:
Recursion completed. All reachable nodes visited.

Visited Nodes:
`['v3', 'v1', 'v2', 'v4', 'v5', 'v7', 'v6']`

Edge Classifications:
- Green: **Tree Edge** (discovered new node)
- Red/Dashed: **Back Edge** (points to ancestor)